

CIENCIAS DE LA COMPUTACIÓN Y PEDAGOGÍA

Tarea 5: Rediseño Curricular

PROPUESTA DE REDISEÑO CURRICULAR

“Synergistic CS Pedagogy: Reestructuración de la Enseñanza de Programación Inicial”

Fundamentación:

Conferencia del Dr. Raj Reddy - “*The Future of AI: Doomers vs. Abundance*”
<https://www.youtube.com/watch?v=ydnOSMbyyQo>

Estudiante: [Nombre del Estudiante]

Asignatura: Fundamentos de Programación y Pensamiento Computacional

Fecha de Entrega: 30 de julio de 2026

Profesor: [Nombre del Profesor]

Institución: [Nombre de la Universidad]

Resumen Ejecutivo

Propuesta de rediseño curricular para la enseñanza de programación inicial, basada en el modelo 70/30 (Meta-Aprendizaje / Aprendizaje Justo a Tiempo), alineada con los principios de la conferencia del Dr. Raj Reddy sobre el futuro de la IA y la educación.

Índice

1. Contexto e Importancia Curricular	2
1.1. El Paradigma del Dr. Raj Reddy	2
1.2. Principios Rectores	2
1.3. Justificación del Rediseño	2
2. Objetivo de la Actividad	3
2.1. Objetivos Específicos	3
3. Estructura del Trabajo a Entregar	3
3.1. Distribución Modular del Syllabus (Modelo 70/30)	3
3.1.1. Descripción del Modelo	3
3.1.2. Tabla de Distribución Semanal	4
3.2. Mecanismos de Evaluación Anti-Plagio y Supervisión Humana	4
3.2.1. Modalidades de Evaluación	4
3.2.2. Rúbrica de Evaluación General	5
3.3. Guía Práctica para Docentes	6
3.3.1. Protocolo de Uso de IA en el Aula	6
3.3.2. Directrices Generales para Docentes	6
3.3.3. Recomendaciones para la Implementación	7
4. Formato del Entregable	7
4.1. Justificación Pedagógica (500 palabras)	8
4.1.1. La Metacognición como Eje del Aprendizaje	8
4.1.2. El Aprendizaje Justo a Tiempo como Complemento	8
4.1.3. El Modelo Maharaja y la Evaluación Anti-Plagio	8
4.1.4. Conclusión	8
5. Conclusiones y Recomendaciones	9
5.1. Conclusiones Principales	9
5.2. Recomendaciones para la Implementación	9
5.3. Reflexión Final	9

1 Contexto e Importancia Curricular

La expansión acelerada de las herramientas de Inteligencia Artificial ha transformado radicalmente el panorama de la educación en ciencias de la computación. Los syllabus tradicionales de programación inicial, que basaban gran parte de su evaluación en la memorización sintáctica y la implementación manual de estructuras simples, han quedado obsoletos frente a la capacidad de los asistentes de IA para generar código funcional en segundos.

1.1 El Paradigma del Dr. Raj Reddy

En su conferencia *“The Future of AI: Doomers vs. Abundance”*, el Dr. Raj Reddy plantea que el aula moderna debe adaptarse a un ecosistema donde la IA provee tutoría continua y respuestas bajo demanda (*Just-in-Time*). Reddy argumenta que el verdadero peligro no radica en la IA en sí, sino en la incapacidad de las instituciones educativas para reinventar sus métodos de enseñanza.

1.2 Principios Rectores

Para evitar la pérdida de rigor académico, las facultades de ingeniería deben estructurar currículos que integren armónicamente dos ejes fundamentales:

- **“Aprender a Aprender”** como eje de pensamiento abstracto y desarrollo de habilidades metacognitivas.
- **“Aprender Justo a Tiempo”** como vehículo de agilidad tecnológica y adaptación a herramientas emergentes.

Esta propuesta curricular se fundamenta en la premisa de que el futuro profesional de la computación no será aquel que memorice sintaxis, sino aquel que sepa formular problemas, evaluar soluciones y adaptarse a entornos tecnológicos cambiantes.

1.3 Justificación del Rediseño

El modelo propuesto responde a tres necesidades críticas:

1. **Relevancia Tecnológica:** Preparar a los estudiantes para un mercado laboral donde la IA es una herramienta cotidiana.
2. **Rigor Académico:** Mantener altos estándares de pensamiento computacional y razonamiento lógico.
3. **Autonomía del Estudiante:** Fomentar la capacidad de autoaprendizaje y adaptación continua.

2 Objetivo de la Actividad

El estudiante, en su rol de diseñador curricular y docente, debe proponer un plan de estudios renovado para la asignatura “**Fundamentos de Programación y Pensamiento Computacional**”, adaptado a la era de la Inteligencia Artificial General (AGI).

2.1 Objetivos Específicos

- Diseñar un syllabus que equilibre el desarrollo de habilidades fundamentales con la integración de herramientas de IA.
- Implementar mecanismos de evaluación que garanticen la integridad académica y el desarrollo del pensamiento crítico.
- Proporcionar directrices claras para el uso pedagógico de la IA en el aula.

3 Estructura del Trabajo a Entregar

La propuesta se desarrolla en los siguientes cuatro componentes principales, detallados en las secciones subsiguientes.

3.1 Distribución Modular del Syllabus (Modelo 70/30)

El diseño académico se estructura en 16 semanas, distribuidas en dos bloques complementarios que se desarrollan de manera concurrente a lo largo del semestre.

3.1.1. Descripción del Modelo

- **70 % Meta-Aprendizaje (*Aprender a Aprender*):**
 - Algoritmos abstractos y fundamentos de lógica booleana.
 - Tipos de Datos Abstractos (TDA) y estructuras de datos fundamentales.
 - Análisis de complejidad algorítmica $O(n)$, $O(n^2)$, $O(\log n)$.
 - Arquitectura de computadores: memoria, CPU, y modelo de Von Neumann.
 - Métodos de autodidaxis y técnicas de estudio metacognitivo.
- **30 % Aprendizaje Justo a Tiempo (JIT):**
 - Integración de frameworks modernos (React, Flask, etc.).
 - Librerías avanzadas para análisis de datos y visualización.
 - Prompting estructurado mediante asistentes virtuales de IA (ChatGPT, Copilot, etc.).
 - Desarrollo de proyectos con asistencia de IA.

3.1.2. Tabla de Distribución Semanal

Semana	70 % Meta-Aprendizaje	30 % Aprendizaje JIT
1	Introducción al Pensamiento Computacional	Configuración de entorno de desarrollo con IA
2	Lógica Booleana y Álgebra de Conjuntos	Introducción al prompting efectivo
3	Algoritmos: Definición y Propiedades	Uso de ChatGPT para depuración de código
4	Tipos de Datos Abstractos: Pilas y Colas	Implementación asistida de TDA
5	Análisis de Complejidad $O(n)$	Optimización con sugerencias de IA
6	Estructuras de Datos: Listas y Arreglos	Uso de bibliotecas estándar con IA
7	Arquitectura de Computadores	Desarrollo de proyectos con IA
8	Examen Parcial (Sin IA)	Revisión y corrección asistida
9	Recursión y Algoritmos Divide y Vencerás	Implementación recursiva con IA
10	Ordenamiento y Búsqueda: Algoritmos Clásicos	Análisis de eficiencia con herramientas de IA
11	Árboles y Grafos: Fundamentos	Visualización de grafos con IA
12	Técnicas de Autodidaxis y Metacognición	Uso de IA para autoaprendizaje
13	Métodos de Optimización y Complejidad	IA para optimización de código
14	Proyecto Final: Diseño Conceptual	Desarrollo del proyecto con IA
15	Revisión y Preparación para Exámenes	Evaluación de proyectos con IA
16	Examen Final (Con defensa oral)	Presentación de proyectos integrales

3.2 Mecanismos de Evaluación Anti-Plagio y Supervisión Humana

Basados en el “Modelo Maharaja” propuesto por el Dr. Reddy, donde el estudiante debe defender su criterio de supervisión, se establecen los siguientes mecanismos de evaluación:

3.2.1. Modalidades de Evaluación

1. Defensa Oral de Código (40 % de la nota final):

- El estudiante presenta un proyecto desarrollado durante el semestre.
- Debe explicar cada decisión de diseño, justificando la elección de algoritmos y estructuras de datos.
- El comité evaluador formula preguntas para verificar la comprensión profunda.
- **Criterio de evaluación:** Claridad expositiva, profundidad conceptual, capacidad de argumentación.

2. Análisis de Código Defectuoso Generado por IA (30 %):

- Se proporciona código generado por IA que contiene errores lógicos, de eficiencia o de seguridad.
- El estudiante debe identificar, diagnosticar y corregir los defectos.
- Debe presentar un informe técnico que explique la naturaleza de cada error.
- **Criterio de evaluación:** Precisión en la detección, calidad de la solución, profundidad del análisis.

3. Refactorización en Tiempo Real (30 %):

- El estudiante recibe un código funcional pero ineficiente.
- En un entorno supervisado (sin acceso a IA), debe refactorizarlo para mejorar su eficiencia y legibilidad.
- Debe explicar en voz alta cada paso del proceso.
- **Criterio de evaluación:** Eficiencia de la solución, manejo del tiempo, capacidad de comunicación.

3.2.2. Rúbrica de Evaluación General

Criterio	Excelente (90-100 %)	Satisfactorio (70-89 %)	Necesita Mejora (<70 %)
Dominio Conceptual	Demuestra comprensión profunda y aplica conceptos con creatividad	Comprende los conceptos fundamentales y los aplica correctamente	Muestra lagunas conceptuales y aplica incorrectamente los conceptos
Razonamiento Lógico	Identifica y resuelve problemas complejos con elegancia	Resuelve problemas estándar sin dificultad	Tiene dificultades para resolver problemas básicos

Continúa en la siguiente página

Criterio	Excelente (90-100 %)	Satisfactorio (70-89 %)	Necesita Mejora (¡70 %)
Análisis de Complejidad	Realiza análisis exhaustivo de complejidad temporal y espacial	Realiza análisis básico de complejidad	No realiza análisis de complejidad o lo hace incorrectamente
Comunicación y Defensa	Explica con claridad y profundidad, respondiendo preguntas con soltura	Se expresa con claridad, pero con limitaciones en preguntas complejas	Dificultades para expresar sus ideas o defender sus decisiones

3.3 Guía Práctica para Docentes

3.3.1. Protocolo de Uso de IA en el Aula

Fase del Aprendizaje	Uso de IA Permitido	Justificación Pedagógica
Evaluación Sumativa	No permitido	Garantiza la medición del desarrollo de habilidades fundamentales
Evaluación Formativa	Permitido	Fomenta el aprendizaje autónomo y la autoevaluación
Proyectos Integradores	Permitido con supervisión	Desarrolla habilidades de colaboración hombre-máquina
Prácticas de Laboratorio	Permitido	Acelera el aprendizaje de herramientas y frameworks
Talleres de Pensamiento Crítico	No permitido	Desarrolla razonamiento independiente y solución de problemas
Desarrollo de Algoritmos	No permitido (inicial) / Permitido (avanzado)	Permite el desarrollo de habilidades fundamentales antes de acelerar el proceso

Cuadro 3: Protocolo de Uso de IA en el Aula

3.3.2. Directrices Generales para Docentes

1. Fase 1: Introducción a Conceptos Fundamentales (Semanas 1-7)

- **IA Permitida:** No en evaluación, sí en tareas formativas.

- **Objetivo:** Desarrollar el pensamiento computacional sin dependencia tecnológica.
 - **Actividades:** Ejercicios de lógica, pseudocódigo, diagramas de flujo.
2. **Fase 2: Desarrollo de Herramientas de Autodidaxis (Semanas 9-13)**
- **IA Permitida:** Sí, como asistente de aprendizaje.
 - **Objetivo:** Enseñar a los estudiantes a utilizar la IA para el autoaprendizaje.
 - **Actividades:** Proyectos de investigación con IA, resolución de problemas con asistencia.
3. **Fase 3: Integración y Síntesis (Semanas 14-16)**
- **IA Permitida:** Sí, en desarrollo de proyectos con supervisión.
 - **Objetivo:** Integrar las habilidades fundamentales con herramientas avanzadas.
 - **Actividades:** Proyectos finales, presentaciones, defensas orales.

3.3.3. Recomendaciones para la Implementación

- **Capacitación Docente:** Los profesores deben recibir formación en el uso pedagógico de la IA.
- **Infraestructura Tecnológica:** Garantizar acceso a herramientas de IA para todos los estudiantes.
- **Ética y Plagio:** Establecer políticas claras sobre el uso de IA y la propiedad intelectual.
- **Evaluación Continua:** Implementar mecanismos de retroalimentación para ajustar el modelo según las necesidades del grupo.

4 Formato del Entregable

El presente documento constituye una propuesta académica formal que incluye:

1. **Syllabus Completo:** Distribución modular detallada en tablas.
2. **Rubros de Evaluación:** Criterios y ponderaciones definidos.
3. **Justificación Pedagógica:** Fundamentación teórica de 500 palabras (presentada a continuación).

4.1 Justificación Pedagógica (500 palabras)

La propuesta curricular aquí presentada se fundamenta en los principios expuestos por el Dr. Raj Reddy en su conferencia “*The Future of AI: Doomers vs. Abundance*”. Reddy plantea que el advenimiento de la Inteligencia Artificial no debe ser visto como una amenaza, sino como una oportunidad para redefinir la educación. El modelo 70/30 propuesto responde a esta visión, equilibrando el desarrollo de habilidades fundamentales con la integración de herramientas tecnológicas avanzadas.

4.1.1. La Metacognición como Eje del Aprendizaje

El 70 % dedicado al *Meta-Aprendizaje* se basa en la premisa de que el pensamiento computacional es una habilidad transferible que trasciende cualquier lenguaje o herramienta específica. Algoritmos abstractos, lógica booleana, tipos de datos abstractos y análisis de complejidad son conceptos que no cambiarán significativamente con el tiempo, a diferencia de las herramientas tecnológicas. Este enfoque prepara a los estudiantes para enfrentar problemas complejos y adaptarse a entornos cambiantes.

4.1.2. El Aprendizaje Justo a Tiempo como Complemento

El 30 % dedicado al *Aprendizaje Justo a Tiempo* reconoce que la velocidad de cambio tecnológico hace imposible enseñar todas las herramientas que un estudiante encontrará en su carrera profesional. En lugar de intentar cubrir un espectro amplio de tecnologías, este modelo enseña a los estudiantes a aprender de manera autónoma, utilizando la IA como catalizador. Como señala Reddy, la capacidad de encontrar y utilizar información bajo demanda es una habilidad esencial en la era de la AGI.

4.1.3. El Modelo Maharaja y la Evaluación Anti-Plagio

Los mecanismos de evaluación propuestos, inspirados en el *Modelo Maharaja* de Reddy, garantizan que el estudiante no pueda eludir el desarrollo de habilidades fundamentales mediante el uso de IA. Las defensas orales y el análisis de código defectuoso evalúan la comprensión profunda y la capacidad crítica, habilidades que no se pueden externalizar a una máquina. La refactorización en tiempo real, por su parte, mide la capacidad de razonamiento y adaptación del estudiante.

4.1.4. Conclusión

Esta propuesta curricular no solo actualiza los contenidos de la enseñanza de programación inicial, sino que transforma el rol del docente de transmisor de conocimiento a facilitador del aprendizaje. Siguiendo las ideas de Reddy, el aula se convierte en un espacio de colaboración entre humanos y máquinas, donde la IA amplifica las capacidades humanas en lugar de reemplazarlas.

5 Conclusiones y Recomendaciones

El rediseño curricular propuesto para la asignatura “*Fundamentos de Programación y Pensamiento Computacional*” representa un paso necesario hacia una educación más relevante, rigurosa y adaptada a los desafíos del siglo XXI.

5.1 Conclusiones Principales

1. **Equilibrio entre Fundamentos y Tecnología:** El modelo 70/30 permite mantener el rigor académico mientras se integran herramientas modernas.
2. **Evaluación Anti-Plagio:** Los mecanismos de defensa oral y análisis de código aseguran la integridad académica.
3. **Flexibilidad Docente:** La guía práctica proporciona directrices claras sin ser restrictiva.
4. **Preparación para el Futuro:** El enfoque en el autoaprendizaje prepara a los estudiantes para un mercado laboral en constante cambio.

5.2 Recomendaciones para la Implementación

- **Fase Piloto:** Implementar el modelo en un grupo reducido antes de escalarlo.
- **Retroalimentación Continua:** Recoger y analizar la opinión de estudiantes y docentes.
- **Actualización Constante:** Revisar el syllabus cada semestre para incorporar nuevas herramientas y metodologías.
- **Colaboración Interdisciplinaria:** Involucrar a expertos en pedagogía y tecnología para mejorar el diseño.

5.3 Reflexión Final

Como bien señala el Dr. Raj Reddy, el futuro de la educación no se trata de competir con la IA, sino de aprovechar su potencial para potenciar el aprendizaje humano. Esta propuesta curricular es un primer paso hacia esa visión, invitando a docentes y estudiantes a embarcarse en un viaje de descubrimiento y transformación.

Referencias

1. Reddy, R. (2024). *The Future of AI: Doomers vs. Abundance*. Conferencia magistral. Disponible en: <https://www.youtube.com/watch?v=ydn0SMbbyyQo>
2. Wing, J. M. (2006). Computational Thinking. *Communications of the ACM*, 49(3), 33-35.

3. Papert, S. (1980). *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books.
4. Bloom, B. S. (1956). *Taxonomy of Educational Objectives*. Longmans, Green and Co.
5. Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.
6. Hazzan, O., & Lapidot, T. (2004). *Constructivist Computer Science Education*. ACM SIGCSE Bulletin, 36(1), 266-270.

Anexos

Anexo A: Ejemplo de Rúbrica Detallada para Defensa Oral

Criterio	Excelente (9-10)	Bueno (7-8)	Satisfactorio (5-6)	Necesita Mejora (¡5)
Profundidad Conceptual	Demuestra dominio excepcional de conceptos	Comprende bien los conceptos fundamentales	Comprende algunos conceptos básicos	Muestra lagunas importantes
Comunicación y Claridad	Se expresa con excepcional claridad y precisión	Se expresa con claridad	Se expresa de manera comprensible	Dificultades significativas de comunicación
Capacidad de Respuesta	Responde preguntas con precisión y profundidad	Responde preguntas correctamente	Responde preguntas básicas	Dificultades para responder preguntas
Organización y Estructura	Presenta ideas de manera lógica y coherente	Presentación bien estructurada	Presentación aceptable	Desorganización en la presentación

Anexo B: Ejemplo de Ejercicio de Refactorización

Enunciado: Se proporciona el siguiente código en Python que calcula la suma de los primeros n números enteros de manera ineficiente. Refactorízelo para mejorar su complejidad temporal y espacial, manteniendo la misma funcionalidad.

```
def suma_numeros(n):
    suma = 0
    for i in range(1, n+1):
```

```
    for j in range(1, i+1):
        suma += 1
    return suma
```

Requerimientos:

- Identificar y explicar los problemas de eficiencia del código original.
- Proponer una implementación alternativa que reduzca la complejidad de $O(n^2)$ a $O(1)$.
- Justificar por qué la nueva implementación es superior.

Anexo C: Ejemplo de Prompting Estructurado para IA

Ejemplo de Prompt para Asistente de IA:

Actúa como un tutor experto en programación. Estoy aprendiendo estructuras de datos y necesito entender el concepto de pilas.

Por favor:

1. Define qué es una pila y sus operaciones fundamentales (push, pop, top).
2. Proporciona un ejemplo en Python de implementación de una pila.
3. Explica un caso de uso práctico en el desarrollo de software.
4. Plantea un ejercicio para que yo resuelva sobre el uso de pilas.